Lab03

Adriana Abreu 53672 Leonor Rebola 53663
53672 53663

March 2026

1 Código CSV

No csv *clientes.csv*, deparámo-nos com idades e nomes em falta e formatação errada no valor dos salários, como podemos ver na seguinte print:

id	nome	idade	salario
1	Ana, Silva		1500.0
2	Pedro	25	
3	Carla, 'Cac	28	2000.5
4	Rui		1200,2
5	Marcia	40	2100.4
6	André, Tes	30	
7	Rodrigo, P		
8	Silvia, Mari	29	1.300,45
9			2000
10	Luis, P	25	1000

Figure 1: clientes.csv com erros

Para tal, procedeu-se às seguintes correções:

- Para os nomes em falta, colocou-se a nomenclatura **N/A**;
- Para as idades em falta, calculou-se a **média das idades** existentes e colocando esse mesmo valor nos espaços em branco;
- Para os salários em falta, calculou-se também a média dos salários já existentes no dataset e corrigiu-se à conversão dos números (1.234,56 para 1234.56)

ID	Nome	Idade	Salario
1	Ana Silva	30	1500.0
2	Pedro	25	1585.94
3	Carla 'Cac:	28	2000.5
4	Rui	30	1200.2
5	Marcia	40	2100.4
6	André Test	30	1585.94
7	Rodrigo P	30	1585.94
8	Silvia Mari:	29	1300.45
9	N/A	30	2000.0
10	Luis P	25	1000.0

Figure 2: clientes.csv limpo

O código usado foi o seguinte:

```python
import csv

input_file = "clientes.csv"
output_file = "clientes_limpo.csv"
log_file = "erros.log"

colunas_esperadas = 4
dados_limpos = []

salarios_validos = []
idades_validas = []


# Fun  es de limpeza
def limpar_texto(texto):
    return texto.strip()


def normalizar_numero(valor):
    valor = valor.strip()

    if valor == "":
        return None

    # Converter n mero europeu para float
    valor = valor.replace(".", "")
    valor = valor.replace(",", ".")
```

```python

        try:
            return float(valor)
        except:
            return None


    def normalizar_inteiro(valor):
        valor = valor.strip()
        if valor.isdigit():
            return int(valor)
        return None


    with open(input_file, "r", encoding="latin-1") as f, open(log_file, "w", encoding="utf-8") as log:

        # Usar DictReader ou csv.reader com aspas
        for numero_linha, linha in enumerate(f, start=1):

            linha = linha.strip()

            # Detectar delimitador
            if linha.count(";") > linha.count(","):
                delimitador = ";"
            else:
                delimitador = ","

            try:
                # Usar csv.reader corretamente para lidar com aspas
                reader = csv.reader([linha], delimiter=delimitador, quotechar='"')
                campos = next(reader)

                if len(campos) != colunas_esperadas:
                    log.write(f"Linha {numero_linha}: n mero errado de colunas -> {linha}\n")
                    continue

                campos = [c.strip() for c in campos]

                id_cliente = campos[0] if campos[0] else "0"
                nome = limpar_texto(campos[1]) if campos[1] else "N/A"

                idade = normalizar_inteiro(campos[2])
                salario = normalizar_numero(campos[3])

```

```python
                # Guardar valores v lidos
                if idade is not None and idade > 0:
                    idades_validas.append(idade)

                if salario is not None and salario > 0:
                    salarios_validos.append(salario)

                dados_limpos.append([id_cliente, nome, idade, salario])

            except Exception:
                log.write(f"Linha {numero_linha}: erro de parsing -> {linha}\n")


    # Calcular m dias
    media_idade = sum(idades_validas) / len(idades_validas) if idades_validas else 0
    media_salario = sum(salarios_validos) / len(salarios_validos) if salarios_validos else 0


    # Substituir valores inv lidos
    for linha in dados_limpos:
        if linha[2] is None or linha[2] <= 0:
            linha[2] = round(media_idade)
        if linha[3] is None or linha[3] <= 0:
            linha[3] = round(media_salario, 2)


    # Guardar CSV limpo
    with open(output_file, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f, quoting=csv.QUOTE_MINIMAL)
        writer.writerow(["ID", "Nome", "Idade", "Salario"])
        writer.writerows(dados_limpos)


    print("Processo conclu do.")
    print("Idade m dia usada:", round(media_idade))
    print("Sal rio m dio usado:", round(media_salario, 2))
    print("Ficheiro limpo criado:", output_file)
    print("Erros registados em:", log_file)
```

2 Código JSON

No documento JSON *produtos.json*, existe campos considerados obrigatórios(id, nome, preco) em falta, inconsistência nos tipos de dados como por exemplo, os preços estarem em texto em vez de valores númericos e por fim inconsistência dos atributos.

```
[
    {
      "id": 101,
      "nome": "Camisola B sica",
      "categoria": "Vestu rio",
      "preco": 19.90,
      "variantes": ["S", "M", "L"]
    },
    {
      "id": 102,
      "nome": "Cal as de Ganga",
      "preco": "39.90",
      "variantes": "32,34,36",
      "descricao": "Tecido azul escuro"
    },
    {
      "id": 103,
      "nome": "Sapatilhas Running",
      "categoria": "Cal ado",
      "preco": 59.9
    },
    {
      "id": 104,
      "nome": "Smartphone X1",
      "preco": 299.00
    },
    {
      "id": 105,
      "nome": "Port til Pro 15",
      "categoria": "Eletr nica",
      "preco": "999",
      "variantes": []
    },
    {
      "id": 106,
      "nome": "Auriculares Bluetooth",
      "categoria": null,
      "preco": "29,90",
      "variantes": ["preto", "branco"]
```

```
    },
    {
      "nome": "Casaco Inverno",
      "categoria": "Vestu rio",
      "preco": 89.90
    }
  ]
```

Para resolver a parte dos campos obrigatórios, utilizando a verificação da existência ou não destes. Se não existir este é adicionado ao documento $log_erros.json$

```
if "id" not in p or "nome" not in p or "preco" not in p:
        erros.append({"produto": p, "erro": "campo obrigat rio em falta"})
        continue
```

Para a inconsistência dos dados fizemos a validação de cada coluna, exceto a do id e nome.

```
    preco = p["preco"]

    if isinstance(preco, str):
        preco = preco.replace(",", ".")
        try:
            preco = float(preco)
        except:
            erros.append({"produto": p, "erro": "preco inv lido"})
            continue

    p["preco"] = preco

    # validar categoria
    if "categoria" not in p or p["categoria"] is None:
        p["categoria"] = "Desconhecida"

    # validar variantes
    if "variantes" in p:
        if isinstance(p["variantes"], str):
            p["variantes"] = p["variantes"].split(",")
        elif not isinstance(p["variantes"], list):
            erros.append({"produto": p, "erro": "variantes inv lido"})
            continue
    else:
        p["variantes"] = []
```

Depois a partir disso adicionamos ao novo documento $produtos_ok.json$